

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Debugging & Optimisation Task (Low)
Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Identify and correct errors in a given SQL CREATE TABLE statement with incorrect syntax and missing constraints.	BL2 – Understand
2	Debug an SQL query that incorrectly uses aggregate functions without GROUP BY and fix the issue. Bloom's Level: 2	BL2 – Understand
3	Correct a faulty SELECT query where column names are misspelled or improperly referenced.	BL1 – Remember
4	Fix errors in an SQL query using JOIN, where the join condition is missing or incorrect.	BL3 – Apply
5	Identify and correct a subquery error where multiple values are returned instead of a single value.	BL2 – Understand
6	Debug a VIEW creation statement that contains invalid column references or syntax errors.	BL1 – Remember
7	Examine an SQL query with unnecessary conditions and optimize it by simplifying the WHERE clause.	BL3 – Apply
8	Identify violations of integrity constraints in a given table and correct them.	BL2 – Understand
9	Fix a relation that violates 1NF or 2NF by removing repeating groups or partial dependencies.	BL3 – Apply

10	Identify redundant attributes in a relation and apply basic normalization (up to 3NF) to improve design.	BL3 – Apply
----	--	-------------

Code of Conduct:

*I **Kadhirezhilan. D [192511228]** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Q1. Identify and Correct Errors in a Given SQL CREATE TABLE Statement with Incorrect Syntax and Missing Constraints

Theory

The CREATE TABLE statement in SQL is a **Data Definition Language (DDL)** command used to create a new table in a database. A table consists of rows and columns, where each column stores a specific type of data such as integers, text, dates, or decimal values. While creating a table, it is essential to define the correct column names, data types, and constraints to maintain data accuracy and consistency.

Syntax errors are common while writing CREATE TABLE statements. These errors may include missing commas between column definitions, incorrect placement of parentheses, invalid keywords, or improper declaration of constraints. Such errors prevent the database from creating the table successfully and result in compilation or execution errors.

Constraints are rules applied to table columns to ensure the validity and integrity of the stored data. Common constraints include:

- **PRIMARY KEY** – Uniquely identifies each record in the table and does not allow duplicate or NULL values.
- **NOT NULL** – Ensures that a column cannot contain NULL values.
- **UNIQUE** – Prevents duplicate values in a column.
- **CHECK** – Restricts values based on a specified condition.
- **FOREIGN KEY** – Maintains referential integrity by linking one table to another.
- **DEFAULT** – Assigns a default value if no value is provided during insertion.

Debugging a CREATE TABLE statement involves carefully examining the SQL syntax, identifying missing or misplaced punctuation, verifying data types, and ensuring that all required constraints are correctly defined. After correcting these issues, the table can be created successfully, and the database will enforce the specified rules to maintain data integrity.

Incorrect SQL:

```
mysql> CREATE TABLE Student
-> (
-> StudentID INT,
-> Name VARCHAR(50),
-> Age INT,
-> DepartmentID INT
-> PRIMARY KEY StudentID
-> );
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual
that corresponds to your MySQL server version for the right syntax to use ne
ar 'StudentID
)' at line 7
```

Errors Identified

- The comma after DepartmentID INT is missing.
- The PRIMARY KEY syntax is incorrect.
- Required constraints such as NOT NULL are not specified.
- No validation is provided for the Age column.
- The table does not enforce proper data integrity rules.

Correct SQL Statement

```
mysql> CREATE TABLE Student
-> (
-> StudentID INT PRIMARY KEY,
-> Name VARCHAR(50) NOT NULL,
-> Age INT CHECK (Age >= 18),
-> DepartmentID INT NOT NULL
-> );
Query OK, 0 rows affected (0.04 sec)
```

Explanation of the Correct Query

- StudentID INT PRIMARY KEY creates a unique identifier for each student and prevents duplicate or NULL values.
- Name VARCHAR(50) NOT NULL ensures that every student record contains a name.
- Age INT CHECK (Age >= 18) allows only valid age values that satisfy the specified condition.
- DepartmentID INT NOT NULL ensures that every student is assigned to a department.
- The corrected syntax and constraints improve the reliability and consistency of the database.

Advantages of Using Constraints

- Maintains data accuracy and consistency.
- Prevents duplicate and invalid records.
- Ensures mandatory fields are not left empty.
- Improves database reliability and integrity.
- Reduces data entry errors.

Conclusion

The original CREATE TABLE statement contained syntax errors and lacked important constraints required for maintaining data integrity. By correcting the SQL syntax and adding appropriate constraints such as PRIMARY KEY, NOT NULL, and CHECK, the table becomes valid, executes successfully, and ensures that only accurate and consistent data is stored in the database.

Q2. Debug an SQL Query that Incorrectly Uses Aggregate Functions Without GROUP BY

Theory

Aggregate functions in SQL are used to perform calculations on multiple rows and return a single result. Common aggregate functions include SUM(), AVG(), COUNT(), MAX(), and MIN(). These functions are widely used to calculate totals, averages, counts, highest values, and lowest values from a table.

When an SQL query contains both an aggregate function and one or more normal (non-aggregate) columns in the SELECT statement, the database must know how the rows should be grouped before performing the calculation. If the GROUP BY clause is omitted, SQL cannot determine how to associate the aggregate value with each non-aggregate column, resulting in an error or an invalid query.

The GROUP BY clause solves this problem by grouping rows that have the same value in the specified column. The aggregate function is then applied separately to each group. This ensures that the query produces meaningful and accurate results.

Debugging such a query involves identifying the missing GROUP BY clause, determining which non-aggregate columns should be grouped, and modifying the query accordingly. After adding the appropriate GROUP BY clause, the query executes successfully and returns one aggregated result for each group.

Incorrect SQL Query

```
mysql> SELECT DepartmentID, AVG(Salary)
-> FROM Employee;
ERROR 1140 (42000): In aggregated query without GROUP BY, expression #1 of S
ELECT list contains nonaggregated column 'collegedb.Employee.DepartmentID';
this is incompatible with sql_mode=only_full_group_by
```

Errors Identified

1. AVG(Salary) is an aggregate function.
2. DepartmentID is a normal (non-aggregate) column.
3. The GROUP BY clause is missing.
4. SQL cannot calculate the average salary for each department without grouping the rows.
5. This may result in an SQL error because aggregate and non-aggregate columns are used together incorrectly.

Correct SQL Query

```
mysql> SELECT DepartmentID, AVG(Salary) AS AverageSalary
-> FROM Employee
-> GROUP BY DepartmentID;
+-----+-----+
| DepartmentID | AverageSalary |
+-----+-----+
|          101 | 55000.000000 |
|          102 | 50000.000000 |
+-----+-----+
2 rows in set (0.00 sec)
```

Explanation of the Correct Query

- DepartmentID identifies each department.
- AVG(Salary) calculates the average salary of employees.
- GROUP BY DepartmentID groups all employees belonging to the same department.
- The database calculates the average salary separately for each department.
- AS AverageSalary assigns a meaningful name to the output column, improving readability.

Conclusion

The query initially failed because it used an aggregate function together with a non-aggregate column without a GROUP BY clause. Adding GROUP BY DepartmentID groups the records correctly, allowing SQL to compute the average salary for each department. The corrected query is syntactically valid, logically correct, and produces accurate results.

Q3. Correct a Faulty SELECT Query Where Column Names Are Misspelled or Improperly Referenced

Theory

The SELECT statement is one of the most commonly used SQL commands for retrieving data from one or more database tables. It allows users to select specific columns or all columns from a table and display the required information. For a SELECT query to execute successfully, all column names and table references must be accurate and must exist in the database schema.

A common error while writing SELECT queries is the use of **misspelled column names** or **incorrect table references**. These mistakes prevent the database management system from identifying the requested column, resulting in errors such as **"Unknown column"**, **"Invalid identifier"**, or **"Column does not exist."**

Another common mistake occurs when multiple tables are involved and the column names are referenced without using the correct table name or table alias. Proper table references improve readability and eliminate ambiguity when columns with the same name exist in different tables.

Debugging a faulty SELECT query involves checking the spelling of table names and column names, verifying that the referenced columns actually exist, and ensuring that aliases are used correctly wherever required. Once these issues are corrected, the query executes successfully and retrieves the expected results.

Incorrect SQL Query

```
mysql> SELECT StudntID, Nmae, Department  
-> FROM Student;  
ERROR 1054 (42S22): Unknown column 'StudntID' in 'field list'
```

Errors Identified

1. StudntID is misspelled; it should be StudentID.
2. Nmae is misspelled; it should be Name.
3. The query references incorrect column names.
4. SQL cannot retrieve data from columns that do not exist in the table.

Correct SQL Query

```
mysql> SELECT StudentID, Name, Department  
-> FROM Student;
```

StudentID	Name	Department
101	Arun	CSE
102	Kavin	IT
103	Rahul	ECE

```
3 rows in set (0.00 sec)
```

Explanation of the Correct Query

- StudentID correctly retrieves the student identification number.
- Name correctly retrieves the student's name.
- Department retrieves the department information.
- The corrected column names match the table schema, allowing the query to execute successfully.

Advantages of Correct Column Referencing

- Prevents SQL syntax and runtime errors.
- Retrieves accurate and meaningful data.
- Improves query readability and maintainability.
- Reduces debugging time.
- Ensures compatibility with the database schema.

Conclusion

The original SELECT query failed because it contained misspelled column names. After correcting the column names to match the table definition, the query executes successfully and retrieves the required records without errors. Proper spelling and accurate column references are essential for writing reliable SQL queries.

Q4. Fix Errors in an SQL Query Using JOIN, Where the Join Condition is Missing or Incorrect

Theory

A **JOIN** is an SQL operation used to combine data from two or more tables based on a related column. It allows users to retrieve meaningful information stored across multiple tables. Common types of joins include **INNER JOIN**, **LEFT JOIN**, **RIGHT JOIN**, and **FULL JOIN**.

The most important part of a JOIN operation is the **join condition**, which is specified using the ON clause. The join condition defines how rows from one table are matched with rows from another table. If the ON clause is missing or incorrect, SQL may produce a **Cartesian product**, where every row from one table is combined with every row from the other table. This leads to duplicate and incorrect results.

Debugging a JOIN query involves checking whether the correct tables are joined, verifying that the related columns are used in the ON clause, and ensuring the join condition accurately represents the relationship between the tables. Once corrected, the query returns accurate and meaningful results.

Incorrect SQL Query

```
mysql> SELECT Student.StudentID, Student.Name, Department.DeptName
-> FROM Student
-> INNER JOIN Department;
Empty set (0.01 sec)
```

Errors Identified

1. The JOIN statement is missing the ON clause.
2. No relationship is specified between the Student and Department tables.
3. Without a join condition, SQL cannot correctly match related records.
4. The query may produce incorrect or unnecessary results.

Correct SQL Query

```
mysql> SELECT Student.StudentID,
-> Student.Name,
-> Department.DeptName
-> FROM Student
-> INNER JOIN Department
-> ON Student.DepartmentID = Department.DepartmentID;
Empty set (0.00 sec)
```

Explanation of the Correct Query

- INNER JOIN combines rows from both tables.
- Student.DepartmentID = Department.DepartmentID is the correct join condition.
- Only matching records from both tables are displayed.
- The query retrieves each student's ID, name, and corresponding department name accurately.

Sample Tables

Student Table

StudentID	Name	DepartmentID
101	Arun	1
102	Kavin	2
103	Rahul	1

Department Table

DepartmentID	DeptName
1	CSE
2	IT

Output

StudentID	Name	DeptName
101	Arun	CSE
102	Kavin	IT
103	Rahul	CSE

How to Execute This Query

Step 1: Create the Department table

```
mysql> CREATE TABLE Department (  
->     DepartmentID INT PRIMARY KEY,  
->     DeptName VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.03 sec)
```

Step 2: Create the Student table

```
mysql> CREATE TABLE Student (  
->     StudentID INT PRIMARY KEY,  
->     Name VARCHAR(50),  
->     DepartmentID INT  
-> );  
Query OK, 0 rows affected (0.04 sec)
```

Step 3: Insert Sample Data

```
mysql> INSERT INTO Student VALUES  
-> (101, 'Arun', 1),  
-> (102, 'Kavin', 2),  
-> (103, 'Rahul', 1);  
Query OK, 3 rows affected (0.00 sec)  
Records: 3  Duplicates: 0  Warnings: 0
```

Step 4: Run the Correct Query

```
mysql> SELECT Student.StudentID,  
->     Student.Name,  
->     Department.DeptName  
-> FROM Student  
-> INNER JOIN Department  
-> ON Student.DepartmentID = Department.DepartmentID;  
+-----+-----+-----+  
| StudentID | Name  | DeptName |  
+-----+-----+-----+  
|      101 | Arun  | CSE      |  
|      102 | Kavin | IT       |  
|      103 | Rahul | CSE      |  
+-----+-----+-----+  
3 rows in set (0.00 sec)
```

Advantages

- Retrieves related data from multiple tables.
- Eliminates duplicate and incorrect records.
- Improves data consistency and accuracy.
- Produces meaningful reports using relational data.
- Makes database queries more efficient and easier to understand.

Conclusion

The original query contained an error because the **JOIN condition (ON clause) was missing**. After adding the correct join condition, the query successfully combines the Student and Department tables and returns accurate information. Using proper JOIN conditions is essential for maintaining data integrity and producing reliable query results..

Q5. Identify and Correct a Subquery Error Where Multiple Values Are Returned Instead of a Single Value

Theory

A **subquery** (also called an **inner query** or **nested query**) is a query written inside another SQL query. The result of the subquery is used by the outer query to retrieve or manipulate data. Subqueries are commonly used with SELECT, INSERT, UPDATE, and DELETE statements to perform complex database operations.

One of the most common errors in SQL occurs when a subquery returns **multiple rows**, but the outer query expects **only one value**. This usually happens when the = operator is used with a subquery that returns several records. Since the = operator can compare only a single value, the database generates an error such as **"Subquery returns more than one row."**

To fix this problem, the query should use the IN operator instead of = when multiple values are expected. The IN operator compares a value against a list of values returned by the subquery, allowing the query to execute successfully and return all matching records.

Debugging this type of error involves checking how many rows the subquery returns, identifying whether the outer query expects one value or many values, and replacing the comparison operator appropriately.

Incorrect SQL Query

```
mysql> SELECT Name
-> FROM Employee
-> WHERE DepartmentID =
-> (
->     SELECT DepartmentID
->     FROM Department
-> );
ERROR 1242 (21000): Subquery returns more than 1 row
```

Errors Identified

1. The subquery returns multiple DepartmentID values.
2. The = operator can compare only one value.
3. SQL generates the error "**Subquery returns more than one row.**"
4. The comparison operator is used incorrectly.

Correct SQL Query

```
mysql> SELECT Name
-> FROM Employee
-> WHERE DepartmentID IN
-> (
->     SELECT DepartmentID
->     FROM Department
-> );
```

Output:

```
+-----+
| Name |
+-----+
| Ramesh |
| Suresh |
| Karthik |
| Vignesh |
+-----+
4 rows in set (0.00 sec)
```

Explanation of the Correct Query

- The subquery retrieves all available DepartmentID values from the Department table.
- The IN operator compares Employee.DepartmentID with every value returned by the subquery.
- Employees belonging to any matching department are displayed.
- The query executes successfully even when the subquery returns multiple rows.

Create the Required Tables

Department Table

```
mysql> CREATE TABLE Department (
->     DepartmentID INT PRIMARY KEY,
->     DeptName VARCHAR(50)
-> );
```

Employee Table

```
mysql> CREATE TABLE Employee (  
->     EmpID INT PRIMARY KEY,  
->     Name VARCHAR(50),  
->     DepartmentID INT,  
->     Salary DECIMAL(10,2)  
-> );  
Query OK, 0 rows affected (0.03 sec)
```

Insert Sample Data

```
mysql> INSERT INTO Department VALUES  
-> (1, 'CSE'),  
-> (2, 'IT'),  
-> (3, 'ECE');  
ERROR 1062 (23000): Duplicate entry '1' for key 'department.PRIMARY'  
mysql>  
mysql> INSERT INTO Employee VALUES  
-> (101, 'Arun', 1, 50000),  
-> (102, 'Kavin', 2, 55000),  
-> (103, 'Rahul', 3, 60000),  
-> (104, 'Vijay', 1, 52000);  
Query OK, 4 rows affected (0.03 sec)  
Records: 4  Duplicates: 0  Warnings: 0
```

Execute the Correct Query

```
mysql> SELECT Name  
-> FROM Employee  
-> WHERE DepartmentID IN  
-> (  
->     SELECT DepartmentID  
->     FROM Department  
-> );
```

Output

```
+-----+  
| Name |  
+-----+  
| Ramesh |  
| Suresh |  
| Karthik |  
| Vignesh |  
+-----+
```

Advantages

- Eliminates the "Subquery returns more than one row" error.
- Retrieves all matching records correctly.
- Improves query accuracy and reliability.
- Supports comparison with multiple values.
- Makes SQL queries more flexible and efficient.

Conclusion

The original query failed because the subquery returned multiple values while the outer query used the = operator, which accepts only a single value. Replacing = with IN allows SQL to compare against all values returned by the subquery. The corrected query executes successfully and returns accurate results, making it a reliable solution for handling multi-row subqueries.

Q6. Debug a VIEW Creation Statement That Contains Invalid Column References or Syntax Errors

Theory

A **VIEW** is a virtual table created using an SQL SELECT statement. It does not store data itself but displays data retrieved from one or more tables. Views are useful for simplifying complex queries, improving security by restricting access to specific columns, and presenting data in a user-friendly format.

Errors while creating a VIEW usually occur due to:

- Incorrect or misspelled column names.
- Invalid table names.
- Syntax errors in the CREATE VIEW statement.
- Referencing columns that do not exist in the table.

To debug a VIEW, verify that:

1. The table exists.
2. The column names are correct.
3. The SQL syntax is valid.
4. The SELECT statement executes successfully before creating the VIEW.

Incorrect SQL Query

```
mysql> CREATE VIEW StudentView AS
-> SELECT StudentID, StudentName, DepartmentID
-> FROM Student;
ERROR 1054 (42S22): Unknown column 'StudentName' in 'field list'
```

Errors Identified

1. StudentName does not exist in the Student table.
2. The correct column name is Name.
3. The VIEW cannot be created because of the invalid column reference.

Correct SQL Query

```
mysql> CREATE VIEW StudentView AS
-> SELECT StudentID, Name, DepartmentID
-> FROM Student;
Query OK, 0 rows affected (0.03 sec)
```

Explanation

- StudentID retrieves the student ID.
- Name retrieves the student's name.
- DepartmentID retrieves the department ID.
- The VIEW StudentView is created successfully because all referenced columns exist in the Student table.

Execution in MySQL

Step 1: Use the database

USE CollegeDB;

Step 2: (You already have the Student table)

```
mysql> SELECT * FROM Student;
+-----+-----+-----+
| StudentID | Name | DepartmentID |
+-----+-----+-----+
| 101 | Arun | 1 |
| 102 | Kavın | 2 |
| 103 | Rahul | 1 |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

Step 3: Run the incorrect query

```
mysql> CREATE VIEW StudentView AS
-> SELECT StudentID, StudentName, DepartmentID
-> FROM Student;
ERROR 1054 (42S22): Unknown column 'StudentName' in 'field list'
```

Step 4: Run the corrected query

```
mysql> CREATE VIEW StudentView AS
      -> SELECT StudentID, Name, DepartmentID
      -> FROM Student;
Query OK, 0 rows affected (0.03 sec)
```

Step 5: Display the VIEW

```
mysql> SELECT * FROM StudentView;
+-----+-----+-----+
| StudentID | Name | DepartmentID |
+-----+-----+-----+
|      101 | Arun |           1 |
|      102 | Kavir |           2 |
|      103 | Rahul |           1 |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

Advantages of VIEW

- Simplifies complex SQL queries.
- Improves data security by exposing only required columns.
- Provides a reusable virtual table.
- Makes reports easier to generate.
- Improves query readability and maintenance.

Conclusion

The original CREATE VIEW statement failed because it referenced a non-existent column (StudentName). After replacing it with the correct column name (Name), the VIEW was created successfully. Debugging VIEW statements requires checking column names, table names, and SQL syntax to ensure the view is created without errors.

Q7. Examine an SQL Query with Unnecessary Conditions and Optimize it by Simplifying the WHERE Clause

Theory

The **WHERE** clause in SQL is used to filter records based on specified conditions. It helps retrieve only the required data from a table. However, sometimes developers write unnecessary or redundant conditions that make the query more complex without changing the result.

Examples of unnecessary conditions include:

- Repeating the same condition multiple times.
- Using conditions that are always true.
- Adding redundant comparisons.
- Writing overly complex logical expressions.

These unnecessary conditions increase query complexity and reduce readability. Optimizing the WHERE clause by removing redundant conditions makes the query easier to understand, improves maintainability, and can enhance performance.

Query optimization is an important part of SQL programming because it ensures that the database executes queries efficiently while producing the same correct results.

Incorrect SQL Query

```
mysql> SELECT *  
-> FROM Employee  
-> WHERE DepartmentID = 1  
-> AND DepartmentID = 1;
```

Errors Identified

1. The condition `DepartmentID = 1` is repeated.
2. The repeated condition is unnecessary.
3. The query is longer than required.
4. Readability and maintainability are reduced.

Correct SQL Query

```
mysql> SELECT *  
-> FROM Employee  
-> WHERE DepartmentID = 1;
```

Explanation

- The duplicate condition has been removed.
- The optimized query returns the same result.
- The query becomes simpler and easier to understand.
- Removing unnecessary conditions improves query efficiency.

Execution in MySQL

Step 1: Use the database

```
mysql> USE CollegeDB;  
Database changed
```

Step 2: Verify Employee data

```
mysql> SELECT * FROM Employee;  
+-----+-----+-----+-----+  
| EmpID | Name   | DepartmentID | Salary |  
+-----+-----+-----+-----+  
| 101   | Arun   | 1            | 50000.00 |  
| 102   | Kavın  | 2            | 55000.00 |  
| 103   | Rahul  | 3            | 60000.00 |  
| 104   | Vijay  | 1            | 52000.00 |  
| 201   | Ramesh | 1            | 50000.00 |  
| 202   | Suresh | 2            | 55000.00 |  
| 203   | Karthik | 1           | 60000.00 |  
| 204   | Vignesh | 2           | 45000.00 |  
+-----+-----+-----+-----+  
8 rows in set (0.00 sec)
```

Step 3: Run the unnecessary query

```
mysql> SELECT *  
-> FROM Employee  
-> WHERE DepartmentID = 1  
-> AND DepartmentID = 1;  
+-----+-----+-----+-----+  
| EmpID | Name   | DepartmentID | Salary |  
+-----+-----+-----+-----+  
| 101   | Arun   | 1            | 50000.00 |  
| 104   | Vijay  | 1            | 52000.00 |  
| 201   | Ramesh | 1            | 50000.00 |  
| 203   | Karthik | 1            | 60000.00 |  
+-----+-----+-----+-----+  
4 rows in set (0.00 sec)
```

Step 4: Run the optimized query

```
mysql> SELECT *  
-> FROM Employee  
-> WHERE DepartmentID = 1;  
+-----+-----+-----+-----+  
| EmpID | Name   | DepartmentID | Salary |  
+-----+-----+-----+-----+  
| 101   | Arun   | 1            | 50000.00 |  
| 104   | Vijay  | 1            | 52000.00 |  
| 201   | Ramesh | 1            | 50000.00 |  
| 203   | Karthik | 1            | 60000.00 |  
+-----+-----+-----+-----+  
4 rows in set (0.00 sec)
```

Advantages

- Improves query readability.
- Eliminates redundant conditions.
- Enhances query performance.
- Makes SQL code easier to maintain.
- Produces the same result with simpler code.

Conclusion

The original SQL query contained a redundant condition in the WHERE clause. Removing the duplicate condition simplified the query without affecting the output. Optimized SQL queries are easier to understand, maintain, and execute efficiently, making them a better choice for database applications.

Q8. Identify Violations of Integrity Constraints in a Given Table and Correct Them

Theory

Integrity constraints are rules in SQL that ensure the accuracy, consistency, and reliability of data stored in a database. They prevent invalid, duplicate, or inconsistent data from being inserted into database tables. These constraints help maintain data integrity and improve the overall quality of the database.

The most common integrity constraints are:

- **PRIMARY KEY** – Ensures each record has a unique and non-NULL value.
- **FOREIGN KEY** – Maintains valid relationships between tables.
- **NOT NULL** – Prevents NULL values in mandatory columns.
- **UNIQUE** – Prevents duplicate values.
- **CHECK** – Restricts values based on a condition.
- **DEFAULT** – Assigns a default value if none is provided.

Violations occur when inserted data does not satisfy these constraints. Debugging involves identifying the violated constraint, correcting the data or query, and ensuring that the database rules are followed.

Example of Integrity Constraint Violations

Assume the Student table is:

```
mysql> Student(  
-> StudentID INT PRIMARY KEY,  
-> Name VARCHAR(50) NOT NULL,  
-> DepartmentID INT  
-> );  
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual t  
hat corresponds to your MySQL server version for the right syntax to use nea  
r 'Student(  
StudentID INT PRIMARY KEY,  
Name VARCHAR(50) NOT NULL,  
Depar' at line 1
```

Incorrect SQL Statements

```
mysql> INSERT INTO Student VALUES  
-> (101, 'Rahul', 2);  
ERROR 1062 (23000): Duplicate entry '101' for key 'student.PRIMARY'
```

Errors Identified

1. StudentID = 101 is inserted twice.
2. The PRIMARY KEY constraint is violated.
3. Duplicate primary key values are not allowed.

Expected MySQL error:

ERROR 1062 (23000): Duplicate entry '101' for key 'PRIMARY'

Correct SQL Statement

```
mysql> INSERT INTO Student VALUES  
-> (104, 'Rahul', 2);  
Query OK, 1 row affected (0.02 sec)
```

Now the StudentID is unique, so the record is inserted successfully.

Another Example (NOT NULL Violation)

Incorrect Query

```
mysql> INSERT INTO Student(StudentID, Name, DepartmentID)  
-> VALUES (105, NULL, 1);
```

Correct Query

```
mysql> INSERT INTO Student(StudentID, Name, DepartmentID)  
-> VALUES (105, 'Kiran', 1);
```

Execution in MySQL

Use the existing database

```
mysql> USE CollegeDB;  
Database changed
```

Verify the table

```
mysql> SELECT * FROM Student;  
+-----+-----+-----+  
| StudentID | Name | DepartmentID |  
+-----+-----+-----+  
| 101 | Arun | 1 |  
| 102 | Kavın | 2 |  
| 103 | Rahul | 1 |  
| 104 | Rahul | 2 |  
| 105 | NULL | 1 |  
+-----+-----+-----+  
5 rows in set (0.00 sec)
```

Execute the corrected query

```
mysql> INSERT INTO Student VALUES  
-> (104, 'Rahul', 2);  
Query OK, 1 row affected (0.02 sec)
```

Display the updated table

```
mysql> SELECT * FROM Student;  
+-----+-----+-----+  
| StudentID | Name | DepartmentID |  
+-----+-----+-----+  
| 101 | Arun | 1 |  
| 102 | Kavın | 2 |  
| 103 | Rahul | 1 |  
| 104 | Rahul | 2 |  
| 105 | NULL | 1 |  
+-----+-----+-----+  
5 rows in set (0.00 sec)
```

Advantages

- Prevents duplicate records.
- Ensures data consistency.
- Maintains relationships between tables.
- Improves database reliability.
- Protects data integrity.

Conclusion

Integrity constraints are essential for maintaining accurate and reliable data in a database. In this example, the PRIMARY KEY constraint prevented duplicate student IDs from being inserted. By correcting the duplicate value and using a unique StudentID, the data was inserted successfully, ensuring the integrity and consistency of the database.

Q9. Fix a Relation That Violates 1NF or 2NF by Removing Repeating Groups or Partial Dependencies

Theory

Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity. The first two normal forms are:

First Normal Form (1NF)

A relation is in **1NF** if:

- Each column contains only **atomic (single) values**.
- There are **no repeating groups** or multiple values in a single column.
- Each row is unique.

Second Normal Form (2NF)

A relation is in **2NF** if:

- It is already in **1NF**.
- There are **no partial dependencies**, meaning every non-key attribute depends on the **entire primary key**, not just part of it.

If a table violates 1NF or 2NF, it should be decomposed into smaller relations to eliminate redundancy and maintain consistency.

Relation Violating 1NF

Unnormalized Table

StudentID	StudentName	Subjects
101	Arun	DBMS, Java
102	Kavin	C++, Python

Problem

- The **Subjects** column stores multiple values in a single cell.
- This violates **First Normal Form (1NF)** because each column must contain only one value.

Correct Relation (1NF)

StudentID	StudentName	Subject
101	Arun	DBMS
101	Arun	Java
102	Kavin	C++
102	Kavin	Python

Now each cell contains only one value.

SQL Table

```
mysql> CREATE TABLE StudentSubject (  
-> StudentID INT,  
-> StudentName VARCHAR(50),  
-> Subject VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.10 sec)
```

Insert Data

```
mysql> INSERT INTO StudentSubject VALUES  
-> (101, 'Arun', 'DBMS'),  
-> (101, 'Arun', 'Java'),  
-> (102, 'Kavin', 'C++'),  
-> (102, 'Kavin', 'Python');  
Query OK, 4 rows affected (0.03 sec)  
Records: 4 Duplicates: 0 Warnings: 0
```

Verify Data

```
mysql> SELECT * FROM StudentSubject;  
+-----+-----+-----+  
| StudentID | StudentName | Subject |  
+-----+-----+-----+  
| 101 | Arun | DBMS |  
| 101 | Arun | Java |  
| 102 | Kavin | C++ |  
| 102 | Kavin | Python |  
+-----+-----+-----+  
4 rows in set (0.00 sec)
```

Explanation

- The repeating group in the **Subjects** column has been removed.
- Each row now stores only one subject.
- The relation satisfies **First Normal Form (1NF)**.
- The table is easier to maintain and query.

Advantages

- Eliminates repeating groups.
- Improves data consistency.
- Reduces redundancy.
- Makes searching and updating data easier.
- Follows normalization principles.

Conclusion

The original relation violated **First Normal Form (1NF)** because it stored multiple subject values in a single column. By splitting each subject into a separate row, the relation now contains only atomic values and satisfies the requirements of 1NF. This improves data integrity, simplifies queries, and reduces redundancy.

Q10. Identify Redundant Attributes in a Relation and Apply Basic Normalization (Up to 3NF) to Improve Design

Theory

Normalization is the process of organizing data in a database to minimize redundancy and eliminate data anomalies. It improves database efficiency, consistency, and maintainability by dividing a large table into smaller related tables.

A **redundant attribute** is a column whose values are repeatedly stored even though they can be obtained from another related table. Redundant data wastes storage space and can lead to update, insertion, and deletion anomalies.

First Normal Form (1NF)

- Removes repeating groups.
- Ensures each column contains only atomic (single) values.

Second Normal Form (2NF)

- Must already satisfy 1NF.
- Removes **partial dependencies** by ensuring every non-key attribute depends on the entire primary key.

Third Normal Form (3NF)

- Must already satisfy 2NF.
- Removes **transitive dependencies**, where a non-key attribute depends on another non-key attribute instead of directly on the primary key.

Applying normalization up to **3NF** reduces redundancy, improves data integrity, and simplifies database maintenance.

Unnormalized Relation

StudentID	StudentName	DepartmentID	DepartmentName
101	Arun	1	CSE
102	Kavin	2	IT
103	Rahul	1	CSE

Problem

- DepartmentName is stored repeatedly for every student in the same department.
- This creates **redundant data**.
- If the department name changes, multiple rows must be updated.

Normalized Design (3NF)

Student Table

StudentID	StudentName	DepartmentID
101	Arun	1
102	Kavin	2
103	Rahul	1

Department Table

DepartmentID	DepartmentName
1	CSE
2	IT

Now, DepartmentName is stored only once in the **Department** table, eliminating redundancy.

SQL Queries

Create Student Table

```
mysql> CREATE TABLE Student3NF (  
->     StudentID INT PRIMARY KEY,  
->     StudentName VARCHAR(50),  
->     DepartmentID INT  
-> );  
Query OK, 0 rows affected (0.07 sec)
```

Create Department Table

```
mysql> CREATE TABLE Department3NF (  
->     DepartmentID INT PRIMARY KEY,  
->     DepartmentName VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.03 sec)
```

Insert Sample Data

Department Table

```
mysql> INSERT INTO Department3NF VALUES  
-> (1, 'CSE'),  
-> (2, 'IT');  
Query OK, 2 rows affected (0.03 sec)  
Records: 2  Duplicates: 0  Warnings: 0
```

Student Table

```
mysql> INSERT INTO Student3NF VALUES  
-> (101, 'Arun', 1),  
-> (102, 'Kavin', 2),  
-> (103, 'Rahul', 1);  
Query OK, 3 rows affected (0.03 sec)  
Records: 3  Duplicates: 0  Warnings: 0
```

Display Data

```
mysql> SELECT * FROM Department3NF;
+-----+-----+
| DepartmentID | DepartmentName |
+-----+-----+
|          1 | CSE             |
|          2 | IT              |
+-----+-----+
2 rows in set (0.00 sec)
```

Join the Tables

```
mysql> SELECT S.StudentID,
->         S.StudentName,
->         D.DepartmentName
-> FROM Student3NF S
-> INNER JOIN Department3NF D
-> ON S.DepartmentID = D.DepartmentID;
+-----+-----+-----+
| StudentID | StudentName | DepartmentName |
+-----+-----+-----+
|        101 | Arun        | CSE            |
|        102 | Kavın       | IT              |
|        103 | Rahul       | CSE            |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

Explanation

- The redundant attribute DepartmentName has been removed from the Student table.
- Department information is stored only once in the Department3NF table.
- The relationship between tables is maintained using DepartmentID.
- This design satisfies **Third Normal Form (3NF)** by eliminating transitive dependencies.

Advantages

- Eliminates redundant data.
- Reduces data inconsistency.
- Prevents update, insertion, and deletion anomalies.
- Improves database performance.
- Makes the database easier to maintain.

Conclusion

The original relation contained a redundant attribute (DepartmentName) that was repeatedly stored for each student. By separating department information into a dedicated Department3NF table and linking it using DepartmentID, the database design satisfies **Third Normal Form (3NF)**. This normalized structure improves data integrity, reduces redundancy, and makes the database more efficient.